

Project Specification Report

Network Simulator

Saniya Dogra | Ankita | Saloni

2023BITE009 | 2023BITE007 | 2023BITE077

Department of Information Technology
National Institute of Technology, Srinagar

1 Introduction

This report documents the design, development, and implementation of a full-stack network simulator constructed as part of the ITL351 Computer Networks Lab semester project. The simulator is written entirely in Python and models all five layers of the TCP/IP protocol stack: Physical, Data Link, Network, Transport, and Application. Rather than treating each layer as an isolated module with a separate menu option, the simulator operates as a unified, sequential pipeline in which data flows downward through the stack during transmission and upward during reception — accurately mirroring real network behaviour.

The project was delivered in three iterative submissions. Submission 1 addressed the Physical and Data Link layers, establishing end-device topologies, hub-based broadcasting, MAC learning switches, error control, and sliding window flow control. Submission 2 added the Network Layer with IPv4/IPv6 addressing, ARP, static routing, and dynamic routing protocols. Submission 3 completed the stack with Transport Layer port management, TCP/UDP segments, Go-Back-N and Selective Repeat protocols, congestion control, and Application Layer services including FTP, Telnet, HTTP, and DNS. Together, the three submissions realise a coherent and educationally faithful simulation of a functional computer network.

2 Objectives

The primary objective of this project is the construction of a comprehensive, parallel-executing network simulator that authentically emulates functional protocol stack mechanics, rather than presenting layer operations as distinct modular options.

- Submission 1 (Layers 1 & 2): Establish end-device topologies alongside functional hubs, bridges, and switches featuring active MAC address learning, parity-based error control, CSMA/CD access control, and sliding window flow control.
- Submission 2 (Layer 3): Implement network routing components including classless IPv4/IPv6 addressing, ARP validation, static route structures, and dynamic routing using RIP, OSPF, BGP, and EIGRP with longest prefix matching.
- Submission 3 (Layers 4 & 5): Enable process-to-process communication via port allocation, TCP and UDP segmentation, Go-Back-N and Selective Repeat ARQ, congestion control, and cooperative application services — FTP, Telnet, HTTP, and DNS.

3 Module Description

The architecture is modular, with decoupled Python files mapping directly to their designated networking layers:

- `physical_layer/` (`connection.py`, `device.py`, `end_device.py`, `hub.py`, `topology.py`, `signal.py`, `line_coding.py`): Manages end-device and hub abstractions, point-to-point and star topology construction, digital signal visualisation, and NRZ/Manchester line coding.
- `datalink_layer/` (`frame.py`, `switch.py`, `bridge.py`, `error_control.py`, `access_control.py`, `flow_control.py`): Houses MAC framing, switch CAM table learning and forwarding, parity-based error detection, CSMA/CD medium access, and Stop-and-Wait / sliding window flow control.
- `network_layer/` (`ip_address.py`, `arp.py`, `ip_packet.py`, `router.py`, `routing_table.py`, `rip.py`, `ospf.py`, `bgp.py`, `eigrp.py`, `subnetting.py`): Controls Layer 3 topology routing, manages ARP tables, runs distance-vector and link-state routing protocols, and performs longest prefix matching.
- `transport_layer/` (`port_manager.py`, `tcp_segment.py`, `udp_segment.py`, `go_back_n.py`, `selective_repeat.py`, `congestion_control.py`, `socket_simulator.py`): Implements port assignment, TCP and UDP header construction, sliding window ARQ, TCP congestion control phases, and socket lifecycle simulation.
- `application_layer/` (`ftp_service.py`, `telnet_service.py`, `http_service.py`, `dns_service.py`, `application_manager.py`): Simulates application environments including FTP file transfer, Telnet remote sessions, HTTP web requests, and DNS name resolution.
- `test_cases/` (`test_physical.py`, `test_datalink.py`, `test_network.py`, `test_transport_application.py`) and `main.py`: Orchestrates all layer demonstrations and the end-to-end full-stack encapsulation test.

4 Working of the System

Data flows sequentially through a double-traversal pipeline — Down-Stack Encapsulation followed by Up-Stack Decapsulation:

- **Application Command Initiation:** The client application (HTTP, FTP, Telnet, or DNS) generates a text payload and passes it down to the Transport Layer.
- **Transport Segmentation:** The TCP module wraps the payload into a segment carrying source port, destination port, sequence number, acknowledgement number, flags, and window size. For lightweight transfers, UDP provides a minimal header with just source port, destination port, and length. Go-Back-N or Selective Repeat manages reliable delivery.
- **Network Addressing and Hop Routing:** An IP packet is constructed with source and destination IP addresses, TTL, and protocol identifier. ARP resolves the next-hop MAC address within the local subnet. If the destination is remote, the routing table is consulted using longest prefix matching, with RIP, OSPF, BGP, or EIGRP providing dynamic route updates.
- **Framing and Bit Streaming:** The Data Link Layer encapsulates the IP packet in an Ethernet frame with source and destination MAC headers. The Physical Layer transmits the resulting bit stream, optionally applying NRZ or Manchester line coding and prepending a synchronisation preamble.

- Medium Transport and Decoding: Bits travel through simulated channels via switches and routers. TTL is decremented at each hop. At the receiver, bits are decoded back into a frame, the IP packet is extracted, the transport segment is delivered to the correct port, and the application payload is handed to the destination process.

5 Layer-by-Layer Functionality

5.1 Physical Layer

The Physical Layer is the foundation of the simulator. The Device base class is extended by EndDevice (a host with a name and MAC address) and Hub (a broadcast repeater). The connect() function in topology.py establishes bidirectional Connection objects between any two devices. When an EndDevice calls send_data(), the data propagates to the connected device; if that device is a Hub, it invokes broadcast() to replicate the signal on all other ports, faithfully modelling hub behaviour. The Signal class visualises bit streams as HIGH/LOW voltage levels, and the LineCoding class provides NRZ and Manchester encoding, giving a concrete representation of wire-level transmission.

5.2 Data Link Layer

The Data Link Layer wraps payloads in Frame objects carrying source MAC, destination MAC, and data fields. The Switch class implements MAC address learning via an AddressTable (CAM table): on each incoming frame the sender MAC is recorded against its port, and the frame is forwarded unicast to the known destination port or flooded to all ports if the destination is unknown. The Bridge class forwards frames between two network segments. Error control is provided by ParityCheck, which appends and verifies an even-parity bit. Medium access control is simulated by CSMACD, which models carrier sensing and collision handling. Flow control is available through Stop-and-Wait at this layer, with Go-Back-N and Selective Repeat also implemented and reused at the Transport Layer.

5.3 Network Layer

The Network Layer manages logical addressing and inter-network routing. IPv4Address and IPv6Address classes store addresses with their subnet masks. The ARPTable maps IP addresses to MAC addresses and simulates broadcast ARP requests and unicast replies. IPPacket and AdvancedPacket encapsulate payloads with full IP header fields including TTL and protocol identifier. The Router class owns multiple RouterInterface instances and a RoutingTable; route lookup uses LongestPrefixMatching to select the most specific matching prefix (CIDR-based LPM). StaticRouting allows manual route entry. Dynamic routing is provided by four protocol classes — RIPProtocol (distance-vector, Bellman-Ford), OSPFProtocol (link-state, Dijkstra), BGPProtocol (path-vector), and EIGRPProtocol (hybrid) — covering the full range of routing protocols examined in the course. Add-on modules for Subnetting and AddressAggregation complete the layer.

5.4 Transport Layer

The Transport Layer provides reliable, process-to-process communication. PortManager maintains a registry of well-known service ports (HTTP: 80, FTP: 21, DNS: 53, HTTPS: 443, Telnet: 23, SSH:

22, SMTP: 25) and dynamically assigns ephemeral ports in the range 49152–65535 for outbound connections. TCPSegment constructs a full TCP header with source port, destination port, sequence number, acknowledgement number, SYN/ACK/FIN flags, and window size. UDPSegment provides the connectionless counterpart. The GoBackN class implements Go-Back-N ARQ: upon detecting a lost frame, all frames from the lost sequence number onward are retransmitted. SelectiveRepeat retransmits only the specific missing frames, improving efficiency. CongestionControl simulates the three TCP congestion phases — Slow Start, Congestion Avoidance, and Fast Retransmit/Recovery. SocketSimulator models the complete BSD socket API lifecycle: socket(), bind(), listen(), accept(), send(), and close().

5.5 Application Layer

The Application Layer provides four simulated services, all launched through the ApplicationManager dispatcher. FTPService simulates an FTP client-server session on port 21, printing connection establishment and file transfer log entries. TelnetService models a remote terminal session on port 23, showing the authentication handshake and command exchange. HTTPService simulates an HTTP/1.1 GET request and 200 OK response on port 80, representing the core web browsing scenario. DNSService simulates domain name resolution on port 53, matching a hostname query against an internal table and returning the corresponding IP address, replicating the recursive DNS lookup process.

6 Key Features

- **Physical Bit Encoding:** Converts string payloads into HIGH/LOW bit-level representations with optional NRZ and Manchester line coding for realistic wire-level simulation.
- **MAC-Based Framing and Unicast Forwarding:** Encapsulates payloads with explicit source and destination MAC fields; switch CAM tables enable unicast forwarding with fallback flooding for unknown destinations.
- **Error and Access Control:** Parity-based error detection at the Data Link Layer and CSMA/CD medium access control provide foundational link reliability.
- **Full Routing Suite:** Static routing, longest prefix matching, and four dynamic routing protocols (RIP, OSPF, BGP, EIGRP) deliver a comprehensive Network Layer.
- **Dual Sliding Window ARQ:** Both Go-Back-N and Selective Repeat are implemented at the Transport Layer, enabling comparison of their retransmission strategies.
- **TCP Congestion Control:** Slow Start, Congestion Avoidance, and Fast Retransmit phases are simulated to model TCP throughput behaviour under network stress.
- **Socket Lifecycle Simulation:** The SocketSimulator models the complete BSD socket API, grounding Transport Layer behaviour in a realistic programming interface.
- **Four Application Services:** FTP, Telnet, HTTP, and DNS services demonstrate end-to-end application-level communication over the full protocol stack.
- **End-to-End Stack Demonstration:** A full encapsulation and decapsulation walkthrough confirms correct data flow through all five layers from application payload to bit stream and back.

7 Assumptions

- All network errors are introduced synthetically within the simulation environment; no real hardware interfaces are used.
- Acknowledgement frames for flow control protocols are assumed to arrive reliably; separate ACK loss tracking is not implemented.
- IP addresses and MAC addresses are assigned statically at device creation time and do not change during a simulation run.
- Routing protocol convergence is modelled by printing representative log output rather than executing iterative Bellman-Ford or Dijkstra computations over time.

8 Limitations

- No real-time packet loss simulation or automated timeout and retransmission recovery is implemented beyond the ARQ protocol demonstrations.
- The Physical Layer is simplified to symbolic HIGH/LOW representations; no electrical signalling or propagation delay is modelled.
- Functions execute sequentially; true parallel transmission across multiple links is not supported.
- The upward decapsulation path in the full-stack demo is logged descriptively rather than being fully parsed at each receiving layer.

9 Future Work

- Add a graphical interface to visualise the network topology and live packet flows across devices.
- Introduce probabilistic noise models and real timeout-based retransmission to simulate packet loss more realistically.
- Extend the upward stack pipeline to fully parse and validate packets at each receiving layer rather than logging descriptive messages.
- Add IPv6 routing, NAT simulation, and DHCP address assignment to extend the Network Layer towards production-level behaviour.

10 Conclusions

The network stack simulator successfully models the deterministic execution mechanics governing data transmission across all five TCP/IP layers. By implementing explicit abstractions from Physical Layer bit encoding and MAC-based framing through Network Layer routing, Transport Layer sliding window protocols, and Application Layer services, the simulator visually demonstrates how a real payload undergoes progressive encapsulation on its journey from an application process to a physical bit stream — and the reverse on reception.

The clear decoupling of layers, combined with a unified `main.py` entry point that runs all demonstrations sequentially, translates textbook protocol theory into an observable, debuggable software architecture. The full-stack encapsulation test confirms that all five layers interact correctly, providing a solid foundation for further extension and experimentation.

11 References

1. Tanenbaum, A. S., & Wetherall, D. J. (2011). *Computer Networks* (5th ed.). Pearson.
2. Kurose, J. F., & Ross, K. W. (2021). *Computer Networking: A Top-Down Approach* (8th ed.). Pearson.
3. ITL351 Course Material — Semester Project Submission 1: Physical and Data Link Layer. NIT Srinagar, February 2026.
4. ITL351 Course Material — Semester Project Submission 2: Network Layer. NIT Srinagar, April 2026.
5. ITL351 Course Material — Semester Project Submission 3: Transport and Application Layer. NIT Srinagar, May 2026.
6. Stevens, W. R., Fenner, B., & Rudoff, A. M. (2003). *UNIX Network Programming, Volume 1: The Sockets Networking API* (3rd ed.). Addison-Wesley.
7. Python Standard Library Documentation — `ftplib` Module. Available at: <https://docs.python.org/3/library/ftplib.html>
8. Python Standard Library Documentation — `telnetlib` Module. Available at: <https://docs.python.org/3/library/telnetlib.html>
9. Anthropic. (2026). Claude (claude-sonnet-4-20250514) [AI language model]. Available at: <https://claude.ai>
10. OpenAI. (2026). ChatGPT (GPT-4o) [AI language model]. Available at: <https://chatgpt.com>

12 Acknowledgment

The authors acknowledge the use of generative artificial intelligence tools, specifically Claude (Anthropic) and ChatGPT (OpenAI), during the development and documentation stages of this project. These AI assistants were leveraged to structure cross-layer verification tests, refine runtime log presentation, format document schemas, and assist in drafting and reviewing the project specification report. All core design decisions, code architecture, and protocol implementations were conceived and executed by the authors.